

Minimum Spanning Tree:

Prim's Algorithm and Prompt Engineering

Samantha Yee

Shriya Manchala

CSSE230-02 Dr. Wollowski

Prim's Algorithm is known as a "Greedy Algorithm" that is primarily used to find the Minimum Spanning Tree (MST) of an undirected and weighted graph. The distinct MST is made up of several edges that connect each of the tree's vertices without forming any cycles within the graph (graph cycles refer to a path that start and end at the same vertex, include an edge, and operate without repeating any edge or vertex (sans the starting/ending vertex)), also ensuring the minimum total edge weight.

The algorithm starts at any arbitrary node within the tree and gradually builds upon the MST by choosing edges with the smallest weight, connecting a node within the MST to a node outside of it (thus growing the MST). By following this pattern, the overall goal of Prim's Algorithm is to connect all nodes within the MST at the lowest cost possible.

In comparison, Kruskal's Algorithm (another greedy algorithm used to find MST's)

utilizes a disjoint set data structure that sorts the tree edges first and then adds them to the MST individually to avoid cycles. Kruskal's Algorithm tends to work better with sparse graphs, where there are a minimal number of edges. Whereas Prim's algorithm tends towards the opposite, working better with dense graphs with more edges to connect adjoining nodes to.

```
public static int primMST(int V, List<List<int[]>> adj) {
    if (V == 0) return 0;

    boolean[] visited = new boolean[V];
    PriorityQueue<int[]> minHeap = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1])); // Min-heap to store edges by weight

    // Start from vertex 0
    visited[0] = true;
    int minCost = 0;
    int edgesUsed = 0;

    // Add all edges from vertex 0 to the priority queue
    for (int[] edge : adj.get(0)) {
        minHeap.offer(edge); // edge = [neighbor, weight]
    }

    while (!minHeap.isEmpty() && edgesUsed < V - 1) {
        int[] edge = minHeap.poll();
        int neighbor = edge[0];
        int weight = edge[1];

        // Skip the edge if the neighbor is already visited
        if (visited[neighbor]) continue;

        // Mark the neighbor as visited and add the weight to minCost
        visited[neighbor] = true;
        minCost += weight;
        edgesUsed++;

        // Add all edges from the current neighbor to the priority queue
        for (int[] nextEdge : adj.get(neighbor)) {
            if (!visited[nextEdge[0]]) {
                minHeap.offer(nextEdge);
            }
        }
    }

    // If not all nodes were connected, return -1 indicating a disconnected graph
    return (edgesUsed == V - 1) ? minCost : -1;
}
```

In our implementation of Prim's algorithm, we primarily utilized a priority queue (min-heap) to determine and select the minimum-weight edge at each step. The priority queue stores edges as pairs of vertices and their weights, allowing constant access to the smallest available edge that connects the growing MST to an unvisited vertex. We represented the varying graphs using an adjacency list structure, implemented as a list of lists containing integer arrays for edges, which enabled efficient traversal of its neighbors. To keep track of visited vertices, we use a boolean array, ensuring constant-time membership checks. Additionally, we utilized an ArrayList to manage a recursive list structure for internal data operations, supported by helper methods within a nested class. This combination of data structures balances fast access, insertion, and removal operations: the priority queue provides insertions and deletions running at $O(\log(V))$, the adjacency list runs at $O(E)$ in terms of edge exploration, and the boolean array runs at $O(1)$ vertex visitation checks. Together, these structures enable our Prim's algorithm to run efficiently with an overall runtime of $O(E * \log(V))$, where V is the number of vertices and E is the number of edges.

Chat-GPT prompt engineering Analysis

Prompt engineering prim's algorithm was a unique experience for building a complex algorithm. For us to start prompt engineering we first needed to understand what our algorithm was and what parts of the algorithm are needed. Once we had the knowledge of the algorithm, we then needed to plan how we would properly ask someone or something like ChatGPT to create code based on our requirements. This required more of a building process as we had to assess what the ChatGPT understood or misunderstood. We started by writing out a basic class with empty methods that we felt were needed based on our understanding. From there, we took our base code and asked for the following:

“Make prim's mst code using these following restrictions: <pasted code>”

```

import java.util.*;

public class PrimsAlgorithm {
    private ArrayList<Integer> t;

    public PrimsAlgorithm() {
        this.t = new ArrayList<>();
    }

    public boolean insert(Integer value) {
        return t.add(value);
    }

    public Integer findMin() {
        if (t.isEmpty()) return null;
        Integer min = t.get(0);
        for (Integer val : t) {
            if (val < min) min = val;
        }
        return min;
    }

    public boolean remove(Integer value) {
        return t.remove(value);
    }

    public int size() {
        return t.size();
    }

    @Override
    public String toString() {
        return t.toString();
    }

    public class Prims {
        private boolean[] visited;
        private PriorityQueue<int[]> pq;
        private int totalWeight;

        public Prims() {
            this.totalWeight = 0;
        }

        public void run(int node, List<List<int[]>> adj) {
            visited[node] = true;

            for (int[] edge : adj.get(node)) {
                int neighbor = edge[0];
                int weight = edge[1];
                if (!visited[neighbor]) {
                    pq.add(new int[]{weight, neighbor});
                }
            }

            while (!pq.isEmpty()) {
                int[] next = pq.poll();
                int weight = next[0];
                int vertex = next[1];

                if (!visited[vertex]) {
                    totalWeight += weight;
                    run(vertex, adj); // recursion
                }
            }

            public int computeMST(int V, List<List<int[]>> adj) {
                visited = new boolean[V];
                pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
                run(0, adj); // start from node 0
                for (boolean v : visited) {
                    if (!v) return -1; // graph was disconnected
                }
                return totalWeight;
            }

            public static int primMST(int v, List<List<int[]>> adj) {
                PrimsAlgorithm algo = new PrimsAlgorithm();
                PrimsAlgorithm.Prims prim = algo.new Prims();
                return prim.computeMST(v, adj);
            }
        }
    }
}

```

The code we received from this was basic and missing a few components. Knowing this, we told ChatGPT to make sure that the code was following the OO design principles. The new code we received was more aligned to proper code design. However, we needed to make sure that the code we currently had would pass different test cases. To correctly prompt engineer good test cases, we asked ChatGPT the following prompt, along with pasting in an example of previous assignment test cases to show the complexity and range of tests needed to ensure our code ran correctly:

“Create java test cases for testing MST Prims Algorithm. Use these test cases as examples of format and complexity to ensure that the code runs at the correct big O runtimes: <pasted code>”

```

import java.util.ArrayList;
import java.util.List;
import java.util.PriorityQueue;

/**
 * Class implementing recursive list operations and Prim's algorithm for MST.
 */
public class PrimAlgorithm {

    private ArrayList<Integer> t;

    public PrimAlgorithm() {
        this.t = new ArrayList<>();
    }

    public boolean insert(Integer value) {
        Prims prims = new Prims();
        prims.insertHelper(t, value);
        return true;
    }

    public Integer findMin() {
        if (t.isEmpty()) return null;
        Prims prims = new Prims();
        return prims.findMinHelper(t, 0, t.get(0));
    }

    public boolean remove(Integer value) {
        Prims prims = new Prims();
        return prims.removeHelper(t, value, 0);
    }

    public int size() {
        Prims prims = new Prims();
        return prims.sizeHelper(t, 0);
    }

    public String toString() {
        Prims prims = new Prims();
        return "[" + prims.toStringHelper(t, 0).toString().trim() + "]";
    }

    public static int primMST(int V, List<List<Integer>>> adj) {
        if (V == 0) return 0;

        boolean[] visited = new boolean[V];
        PriorityQueue<int[]> minHeap = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1])); // Min-heap to store edges by weight

        // Start from vertex 0
        visited[0] = true;
        int minCost = 0;
        int edgesUsed = 0;

        // Add all edges from vertex 0 to the priority queue
        for (int[] edge : adj.get(0)) {
            minHeap.offer(edge); // edge = [neighbor, weight]
        }

        while (!minHeap.isEmpty() && edgesUsed < V - 1) {
            int[] edge = minHeap.poll();
            int neighbor = edge[0];
            int weight = edge[1];

            // Skip the edge if the neighbor is already visited
            if (visited[neighbor]) continue;

            // Mark the neighbor as visited and add the weight to minCost
            visited[neighbor] = true;
            minCost += weight;
            edgesUsed++;

            // Add all edges from the current neighbor to the priority queue
            for (int[] nextEdge : adj.get(neighbor)) {
                if (!visited[nextEdge[0]]) {
                    minHeap.offer(nextEdge);
                }
            }
        }

        // If not all nodes were connected, return -1 indicating a disconnected graph
        return (edgesUsed == V - 1) ? minCost : -1;
    }

    public class Prims {

        // Recursive helper methods for list operations

        public void insertHelper(ArrayList<Integer> list, Integer value) {
            insertRecursive(list, value, 0);
        }

        private void insertRecursive(ArrayList<Integer> list, Integer value, int index) {
            if (index == list.size()) {
                list.add(value);
                return;
            }
            insertRecursive(list, value, index + 1);
        }

        public Integer findMinHelper(ArrayList<Integer> list, int index, int currentMin) {
            if (index == list.size()) {
                return currentMin;
            }
            if (list.get(index) < currentMin) {
                currentMin = list.get(index);
            }
            return findMinHelper(list, index + 1, currentMin);
        }

        public boolean removeHelper(ArrayList<Integer> list, Integer value, int index) {
            if (index == list.size()) {
                return false;
            }
            if (list.get(index).equals(value)) {
                list.remove(index);
                return true;
            }
            return removeHelper(list, value, index + 1);
        }

        public int sizeHelper(ArrayList<Integer> list, int index) {
            if (index == list.size()) {
                return 0;
            }
            return 1 + sizeHelper(list, index + 1);
        }

        public StringBuilder toStringHelper(ArrayList<Integer> list, int index) {
            StringBuilder sb = new StringBuilder();
            if (index == list.size()) {
                return sb;
            }
            sb.append(list.get(index));
            if (index < list.size() - 1) {
                sb.append(", ");
            }
            sb.append(toStringHelper(list, index + 1));
            return sb;
        }
    }
}

```

The test cases that we received from ChatGPT resulted in cases that had the same structure and rigor as the example we provided, including the point system and assurance of its runtime expectancy. To enhance the test cases, we made sure to prompt ChatGPT to remove certain code that was written such as classes and recursion. In doing so we isolate the recursion and bulk of the code inside the main `prims` class.

```

import org.junit.Test;
import org.junit.AfterClass;
import static org.junit.Assert.*;
import java.util.*;

public class Testing {

    private static int points = 0;

    // ----- List Operation Tests -----

    @Test
    public void testInsertAndToString() {
        PrimsAlgorithm prims = new PrimsAlgorithm();
        prims.insert(10);
        prims.insert(20);
        prims.insert(5);
        assertEquals("[10, 20, 5]", prims.toString());
        points += 5;
    }

    @Test
    public void testFindMin() {
        PrimsAlgorithm prims = new PrimsAlgorithm();
        prims.insert(15);
        prims.insert(3);
        prims.insert(9);
        assertEquals((Integer) 3, prims.findMin());
        points += 5;
    }

    @Test
    public void testRemove() {
        PrimsAlgorithm prims = new PrimsAlgorithm();
        prims.insert(7);
        prims.insert(8);
        assertTrue(prims.remove(7));
        assertFalse(prims.remove(100));
        assertEquals("[8]", prims.toString());
        points += 5;
    }

    @Test
    public void testSize() {
        PrimsAlgorithm prims = new PrimsAlgorithm();
        assertEquals(0, prims.size());
        prims.insert(1);
        prims.insert(2);
        prims.insert(3);
        assertEquals(3, prims.size());
        points += 5;
    }
}

```

```

@Test
public void testInsertDuplicates() {
    PrimsAlgorithm prims = new PrimsAlgorithm();
    prims.insert(5);
    prims.insert(5);
    prims.insert(5);
    assertEquals("[5, 5, 5]", prims.toString());
    points += 3;
}

@Test
public void testRemoveAllElements() {
    PrimsAlgorithm prims = new PrimsAlgorithm();
    prims.insert(3);
    prims.insert(4);
    prims.insert(5);
    prims.remove(3);
    prims.remove(4);
    prims.remove(5);
    assertEquals("[]", prims.toString());
    assertEquals(0, prims.size());
    points += 3;
}

@Test
public void testFindMinWithDuplicates() {
    PrimsAlgorithm prims = new PrimsAlgorithm();
    prims.insert(2);
    prims.insert(2);
    prims.insert(2);
    assertEquals((Integer) 2, prims.findMin());
    points += 3;
}

@Test
public void testRemoveFromEmptyList() {
    PrimsAlgorithm prims = new PrimsAlgorithm();
    assertFalse(prims.remove(1));
    assertEquals("[]", prims.toString());
    points += 3;
}

@Test
public void testFindMinOnEmptyList() {
    PrimsAlgorithm prims = new PrimsAlgorithm();
    assertNull(prims.findMin());
    points += 3;
}

```

```
// ----- MST Algorithm Tests -----
```

```
@Test
public void testBasicTriangleGraph() {
    int V = 3;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    adj.get(0).add(new int[]{1, 1}); adj.get(1).add(new int[]{0, 1});
    adj.get(1).add(new int[]{2, 2}); adj.get(2).add(new int[]{1, 2});
    adj.get(0).add(new int[]{2, 3}); adj.get(2).add(new int[]{0, 3});
    assertEquals(3, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}
```

```
@Test
public void testDisconnectedGraph() {
    int V = 4;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    adj.get(0).add(new int[]{1, 1}); adj.get(1).add(new int[]{0, 1});
    adj.get(2).add(new int[]{3, 2}); adj.get(3).add(new int[]{2, 2});
    assertEquals(-1, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}
```

```
@Test
public void testLargeSparseGraph() {
    int V = 10000;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    for (int i = 0; i < V - 1; i++) {
        adj.get(i).add(new int[]{i + 1, 1});
        adj.get(i + 1).add(new int[]{i, 1});
    }
    long start = System.currentTimeMillis();
    int mstCost = PrimsAlgorithm.primMST(V, adj);
    long end = System.currentTimeMillis();
    assertEquals(V - 1, mstCost);
    assertTrue("Expected runtime < 2s", (end - start) < 2000);
    points += 5;
}
```

```
@Test
public void testMediumGraph() {
    int V = 900;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    for (int i = 0; i < V - 1; i++) {
        adj.get(i).add(new int[]{i + 1, 2});
        adj.get(i + 1).add(new int[]{i, 2});
    }
}
```

```
    }
    assertEquals((V - 1) * 2, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}
```

```
@Test
public void testCompleteSmallGraph() {
    int V = 5;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            adj.get(i).add(new int[]{j, 1});
            adj.get(j).add(new int[]{i, 1});
        }
    }
    assertEquals(4, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}
```

```
@Test
public void testGraphWithNegativeWeights() {
    int V = 3;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    adj.get(0).add(new int[]{1, -2}); adj.get(1).add(new int[]{0, -2});
    adj.get(1).add(new int[]{2, -3}); adj.get(2).add(new int[]{1, -3});
    assertEquals(-5, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}
```

```
@Test
public void testMediumDenseGraph() {
    int V = 300;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < Math.min(V, i + 4); j++) {
            adj.get(i).add(new int[]{j, 1});
            adj.get(j).add(new int[]{i, 1});
        }
    }
    assertEquals(V - 1, PrimsAlgorithm.primMST(V, adj));
    points += 5;
}
```

```
@Test
public void testSmallDisconnectedGraph() {
    int V = 6;
    List<List<int[]>> adj = new ArrayList<>();
}
```

```

List<List<int[]>> adj = new ArrayList<>();
for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
adj.get(0).add(new int[]{1, 3});
adj.get(1).add(new int[]{0, 3});
adj.get(2).add(new int[]{3, 2});
adj.get(3).add(new int[]{2, 2});
assertEquals(-1, PrimsAlgorithm.primMST(V, adj));
points += 3;
}

@Test
public void testIrregularGraph() {
    int V = 7;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    adj.get(0).add(new int[]{1, 4});
    adj.get(0).add(new int[]{2, 3});
    adj.get(1).add(new int[]{0, 4});
    adj.get(2).add(new int[]{0, 3});
    adj.get(2).add(new int[]{3, 2});
    adj.get(3).add(new int[]{2, 2});
    adj.get(3).add(new int[]{4, 6});
    adj.get(4).add(new int[]{3, 6});
    adj.get(4).add(new int[]{5, 5});
    adj.get(5).add(new int[]{4, 5});
    adj.get(5).add(new int[]{6, 1});
    adj.get(6).add(new int[]{5, 1});
    assertEquals(21, PrimsAlgorithm.primMST(V, adj));
    points += 5;
}

@Test
public void testStarGraph() {
    int V = 5;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    for (int i = 1; i < V; i++) {
        adj.get(0).add(new int[]{i, i});
        adj.get(i).add(new int[]{0, i});
    }
    assertEquals(10, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}

@Test
public void testRingGraph() {
    int V = 6;
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    for (int i = 0; i < V; i++) {

```

```

        int next = (i + 1) % V;
        adj.get(i).add(new int[]{next, 1});
        adj.get(next).add(new int[]{i, 1});
    }
    assertEquals(5, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}

@Test
public void testSingleNode() {
    int V = 1;
    List<List<int[]>> adj = new ArrayList<>();
    adj.add(new ArrayList<>());
    assertEquals(0, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}

@Test
public void testEmptyGraph() {
    int V = 0;
    List<List<int[]>> adj = new ArrayList<>();
    assertEquals(0, PrimsAlgorithm.primMST(V, adj));
    points += 3;
}

@AfterClass
public static void printPoints() {
    System.out.println("Total Points: " + points + "/100");
}
}

```

Other requirements we prompted ChatGPT were to only create recursive code for certain methods such as insert, remove, and others, naming helper methods similar to insertHelper(), and keeping the code in line with OO design principles. We also made sure that it created a large number of test cases in varying sizes and complexity to ensure that our implementation code works correctly. Once we were given the code, we ran the program with our test cases and found that all cases passed quickly.

Runtime Analysis: (table w/ sample calculations and analysis)

Method Name:	Sample Input Size (n):	Time Complexity:
Insert()	1000	$O(n)$
findMin()	1000	$O(n)$
Remove()	1000	$O(n)$
Size()	1000	$O(n)$
toString()	1000	$O(n^2)$
primMST(V, adj)	1000	$O(E * \log(V))$

Case Analysis:

Case	Big-O Complexity	Typical Example
Best Case	$O(E * \log V)$	Tree or line graph
Average Case	$O(E * \log V)$	Mesh, star, or grid
Worst Case	$O(V^2 * \log V)$	Complete graph

Runtime Summary:

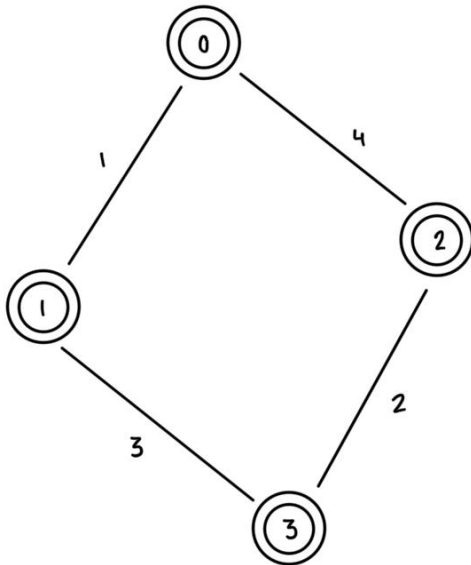
Function	Input Size	Est. Runtime	Scalability
insert()	$n = 10^4$	$O(n) = 10^4$	Efficient
toString()	$n = 10^4$	$O(n^2) = 10^8$	Expensive!
primMST()	$V = 10^4$	$E * \log V \approx 130K$	Scales well (sparse)
primMST()	$V = 100$	$E * \log V \approx 35K$	Costly (dense)

Big-O Runtime Analysis:

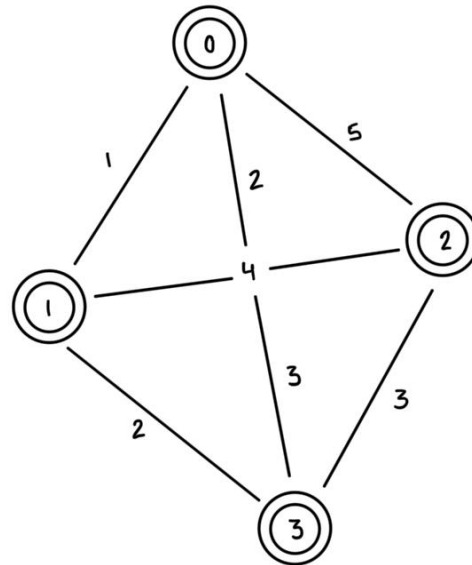
After analyzing and researching what the runtimes of each method should be, we plugged our analysis table into ChatGPT to see how it compares the runtimes of our time complexity. To make sure that we assess large scales of data, we set the sample input sizes to 1000. Overall, the scalability that was calculated based on our runtime analysis data matched with what we expected to show. Prim's algorithm is usually better to work with sparse graphs over dense graphs, hence why the Kruskal's algorithm is better for handling dense graphs. Based on our ArrayList and Heap structures, the Big O runtimes and overall performance is roughly around where we expected ChatGPT to perform.

Graph Diagrams:

Small sparse graph:



Small Fully Connected Graph:

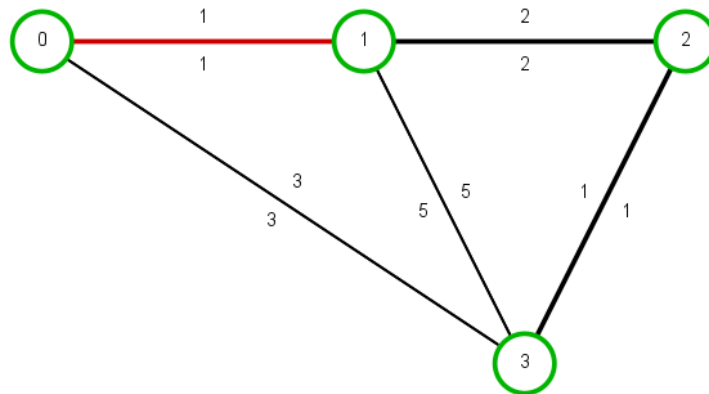


Additional Commentary:

Throughout the development of this project, we found that the way in which we phrased certain prompts directly impacted the quality of work we received. This was extremely evident when we attempted to create an MST visualizer software that would've allowed us to simulate the MST utilizing our code in real time. The prompt we utilized to generate this outcome was:

“Create a graph visualizer program utilizing the following Prim’s MST code <insert PRIMS program code>”

This method was intended to be an integral part of our testing process, however during our initial attempts to create the software we found that ChatGPT was only capable of creating “simulation software” that took a predetermined set of edges/ nodes and created the MST according to our code (the following image demonstrates the initial attempt).



However, we'd intended to create a visualizer in which we could insert and remove edges (with varying edge weight that the user could determine) and be able to see the MST sort the edges in real time as that would've optimized testing and bug fixing outcomes. We attempted to reach this outcome utilizing the following prompt:

"I need you to create a visualizer algorithm that works with my Prim's MST code and allows me to insert and remove nodes/ edges and simulate a prim's algorithm in real time <insert Prim'sMST code> <insert previous visualizer code>"

The visualizer code that we achieved with this prompt had changed the visualizer in the following ways, it created buttons to insert and remove nodes, a meter to change the edge weight of inserted edges, a run function to run the MST upon the inserted edges, and a clear function. However in terms of the functionality the visualizer was unable to call the specific methods that would allow it to actually insert and remove the nodes. We attempted to fix this by utilizing a more specific prompt:

"I need the visualizer to work alongside my code, any insert and remove functions called by pushing the respective buttons in the visualizer, need to call the respective insert and remove methods in my code to simulate the MST implementation utilize this visualizer code from a BST implementation as an example of which <insert BST visualizer code>"

Even with a more specific prompt as well as an example of a functional visualizer the code we received was non-functional (the following image demonstrates the second as well as the third attempts).

Ultimately, through this experience we found that although ChatGPT was capable of creating original code and implementing distinct methods that work independently, it was harder to create code that was meant to work alongside other, existing code, without the AI being confused by the various implementations. We found that ChatGPT responds well to very detailed direction, and struggled when faced with the prospect of making decisions.